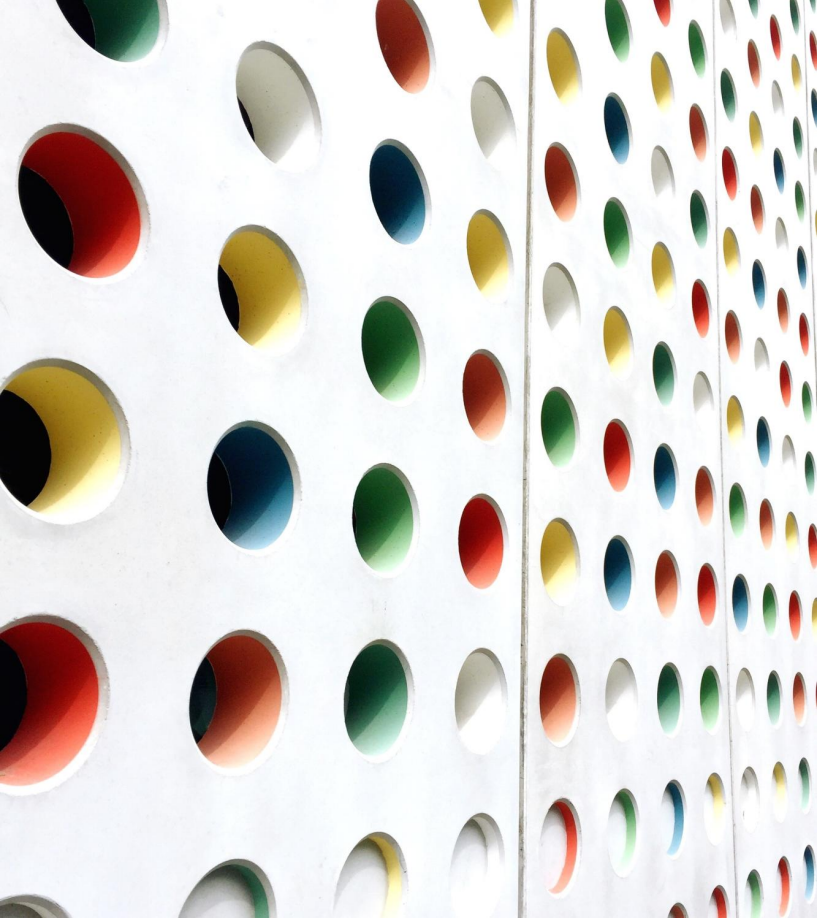




Chapter 1

SIT DOLOR AMET

Chapter 1: Overview of Software Engineering

1.1. Overview of Software Engineering

1.2. SDLC models

1.3. Requirement Engineering

1.3.1. Types of Requirements: -Functional and Non-functional

1.3.2. Four Phases of Requirement Engineering

1.4. Software requirement Specification (SRS)

1.4.1. Structure and contents of SRS

1.4.2. IEEE SRS Format Case studies : based on SRS

Overview of Software Engineering

1. What is Software Engineering?

Software Engineering = systematic approach to **designing, developing, testing, deploying, and maintaining** software.

Combines principles of **Computer Science + Engineering practices**.

Aim: Deliver **high-quality, reliable, maintainable, cost-effective** software within time.

Need for Software Engineering

Increasing **complexity** of software.

High **cost** of failures (banking, healthcare, defense).

Need for **scalability & maintainability**.

Managing **large teams** and distributed development.

Characteristics of Good Software

Correctness

Reliability

Efficiency

Maintainability

Usability

Scalability



Applications of Software Engineering

Business Systems (ERP, CRM, banking software).

Embedded Systems (automobiles, IoT devices).

Real-time Systems (air traffic control, defense).

Web & Mobile Applications.

AI & Data Science Applications.



2. Requirement Engineering

2.1. Types of Requirements – Functional and Nonfunctional

2.2. Four Phases of Requirement Engineering

2.3. Software requirement Specification (SRS)

2.3.1. Structure and contents of SRS

2.3.2. IEEE standard format for SRS

Software Requirements

The [Software Requirements](#) are description of features and functionalities of the target system.

Requirements [convey the expectations of users](#) from the software product.

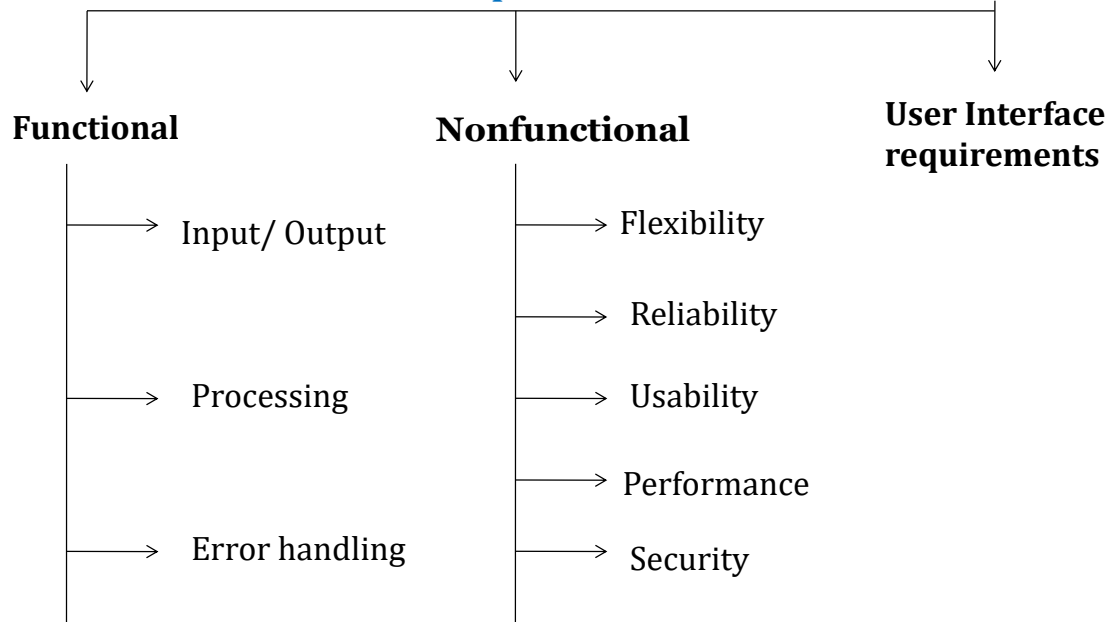
The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view.

Requirement Engineering

The process to gather the software requirements from client, analyze and document them is known as [requirement engineering](#).

The [goal of requirement engineering](#) is to develop and maintain sophisticated and descriptive 'System Requirements Specification' document.

Requirements



Functional Requirements

Requirements, which are related to functional aspect of software fall into this category.

They define functions and functionality within and from the software system.

Example:

1. Search option given to user to search from various invoices.

2. User should be able to mail any report to management.

3. Users can be divided into groups and groups can be given separate rights.

4. Should comply business rules and administrative functions.

Non-Functional Requirements

They are implicit or expected characteristics of software, which users make assumption of.

Security

Performance

Safety

Flexibility

Reliability

Usability

Non-Functional requirements:

Performance Requirements

Performance of the system should be **fast and accurate**.

- – System shall handle **expected and unexpected errors**.
- – Should be able to **handle large amount of data**.

Safety Requirements

- Must be two servers **one main server and one backup server**.

Security Requirements

- – User authentication and validation of members using their unique member ID.
- – Proper accountability which include **not allowing a member to see other members account**.
- – **CAPTCHA words** will be used for user login.

Non-Functional requirements

Ease of use:

A simple but quality user interface should be developed to make it easy to understand and required less training.

Error handling:

- Error should be considerably minimized and an appropriate error message that guides the user to recover from an error should be provided.

Business Rules:

- Only user with the role admin can generate business reports

Software Quality Attributes

The Quality of the database is maintained in such a way so that it can be very user friendly to all the users of the database

User Interface requirements

UI is an important part of any software or hardware system. A software is widely accepted if it is -

- easy to operate
- quick in response
- effectively handling operational errors
- providing simple yet consistent user interface

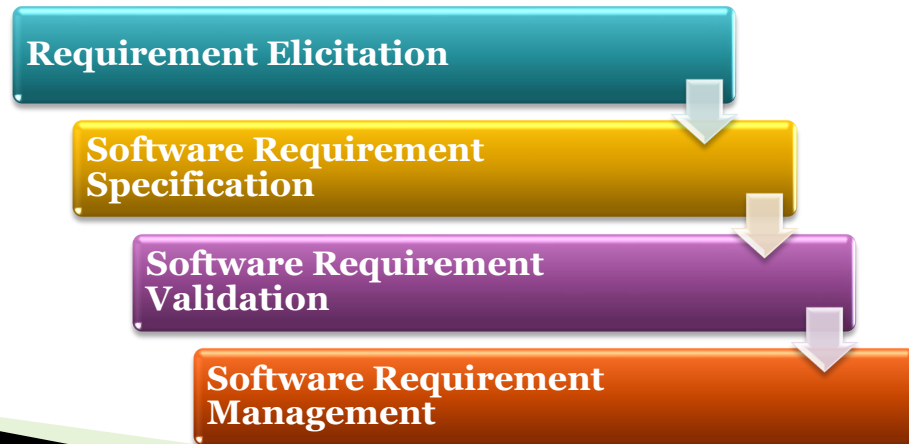
User acceptance majorly depends upon how user can use the software.



Four Phases of Requirement Engineering

- ▶ **Requirements Engineering (RE)** refers to the process of defining, documenting, and maintaining requirements in the engineering design process.

- ▶ **Phases of Requirement Engineering**



OOSE-by Ms. Asmita Choudhari IICMR MCA

Requirement Elicitation

Requirement elicitation process can be depicted using the following diagram:



Requirements gathering - The developers discuss with the client and end users and know their expectations from the software.

Organizing Requirements - The developers **prioritize and arrange the requirements** in order of importance, urgency and convenience.

Negotiation & discussion - If requirements are ambiguous or there are some conflicts in requirements of various stakeholders, it is then negotiated and discussed with stakeholders.

The requirements come from various stakeholders. To remove the ambiguity and conflicts, they are discussed for clarity and correctness.

Documentation - All formal & informal, functional and non-functional requirements are documented and made available for next phase processing.

Software Requirement Validation

After requirement specifications are developed, the requirements mentioned in this document are validated. User might ask for illegal, impractical solution or experts may interpret the requirements in

Requirements can be checked against following conditions -

If they can be practically implemented

If they are valid and as per functionality and domain of software

If there are any ambiguities

If they are complete

If they can be demonstrated correctly.

Requirement management

Requirement management **is the process of managing changing requirements during the requirements engineering process and system development.**

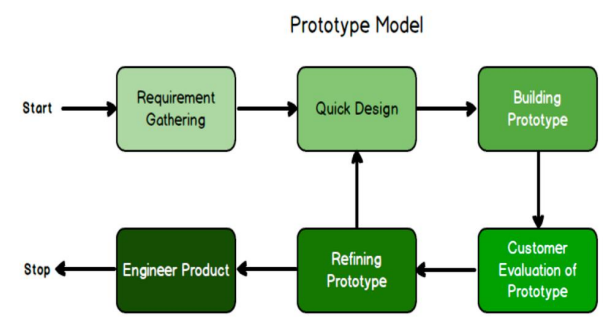
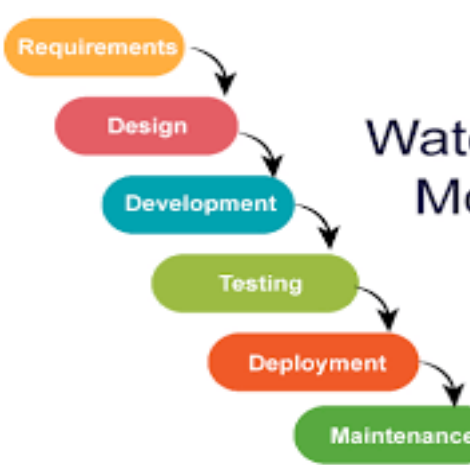
New requirements emerge during the process as business needs a change, and a better understanding of the system is developed.

The priority of requirements from different viewpoints changes during development process.

The business and technical environment of the system changes during the development.

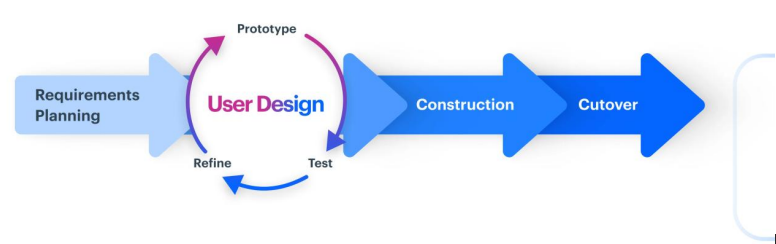
SDLC MODELS

Model	What it is	When to Use	Why to Use	Examples / Suitable Systems
Waterfall	Linear, sequential phases (Req → Design → Code → Test → Deploy).	Requirements are clear, stable, and well-documented.	Simple, easy to manage, clear deliverables.	Payroll system, banking system, government software.
Spiral	Iterative model with risk analysis in each cycle.	Large, complex, high-risk projects.	Handles risks early, combines prototyping + waterfall.	Defense projects, aerospace, R&D systems.
Prototyping	Build working prototype first, refine based on feedback.	Requirements are unclear or users unsure of needs.	Clarifies requirements, reduces late changes.	Web apps, mobile apps, UI/UX heavy apps.
RAD (Rapid App Dev)	Fast, iterative dev with prototyping & component reuse.	Need quick delivery, scope is limited, requirements known.	Saves time, involves customer feedback.	Small business apps, internal tools, short-deadline systems.
RUP (Rational Unified Process)	Iterative, object-oriented process with UML (Phases: Inception → Elaboration → Construction → Transition).	Large, enterprise-level projects with changing requirements.	Structured, supports OO design, iterative and flexible.	Enterprise apps, distributed systems, long-term corporate projects.

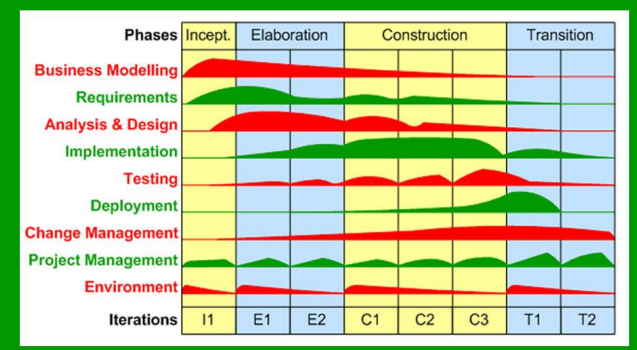


kissflow

Rapid Application Development (RAD)



THE RATIONAL UNIFIED PROCESS (RUP)



Need for SRS

People who are interested in new system? -Client, developer & end user.

SRS establishes basis for agreement between the user(client) and the supplier(development team) on what the s/w product will do.

- Users needs have to be satisfied, but user may not understand software development process**
- Developers will develop the system, but may not know about problem domain**

Need for SRS

SRS is the medium to bridge the communication gap and specify user needs in a manner both can understand

SRS provides a reference for validation of the final product

- Clear understanding about what is expected.
- Validation - “ S/W satisfies the SRS “

Structure/ Format of an SRS

